

line. In this chapter, we will examine the concepts involved with working with databases using Laravel framework.

Getting Started

The main highlight of Laravel is that the framework provides support for MySQL, PostgreSQL, SQLite and Microsoft SQL Server. Firstly, you should locate the database configuration file at `config/database.php` and define your database connections. You should specify which connection should be used by the app by default. The `database.php` file can be used in combination with Laravel Homestead virtual machine, using which you can work with the language from within your local machine.

Working with SQLite

SQLite is a popular relational database management system included with the C programming library. The database is embedded into the program instead of a client-server approach. You have to first create a new SQLite database by using the following command:

```
touch database/datasample.sqlite
```

The next step is to configure environment variables in such a way that it points to the new database as shown below:

```
DB_CONNECTION=sqlite
```

```
DB_DATABASE=/absolute/path/to/datasample.sqlite
```

To perform database operations via Laravel, you need to make use of the three keys such as read, write and sticky. The read and write keys accept host key in the form of an array. For local connections, you should use local-level LAN IP address, which would start with 192.168.x.x.



Establish connectivity with Laravel application using SQLite

However, if you would like to deploy the app on a production server, the host values will be the IP of the database server. The sticky portion is optional and if a write operation is performed after setting it to true, then all read operations in future will make use of the write connection. You should also set relevant values pertaining to host IP, driver, database, username, password and other parameters as shown in the code listing below.